Universidade do Minho

Ano Letivo: 2025/26

Mineração de Dados

Text Mining & NLP III

Miguel Rocha | mrocha@di.uminho.pt
Diana Ferreira | diana@di.uminho.pt

Contents

1 Word Embeddings

2 Transformers

3 Contextual Models (BERT, GPT)

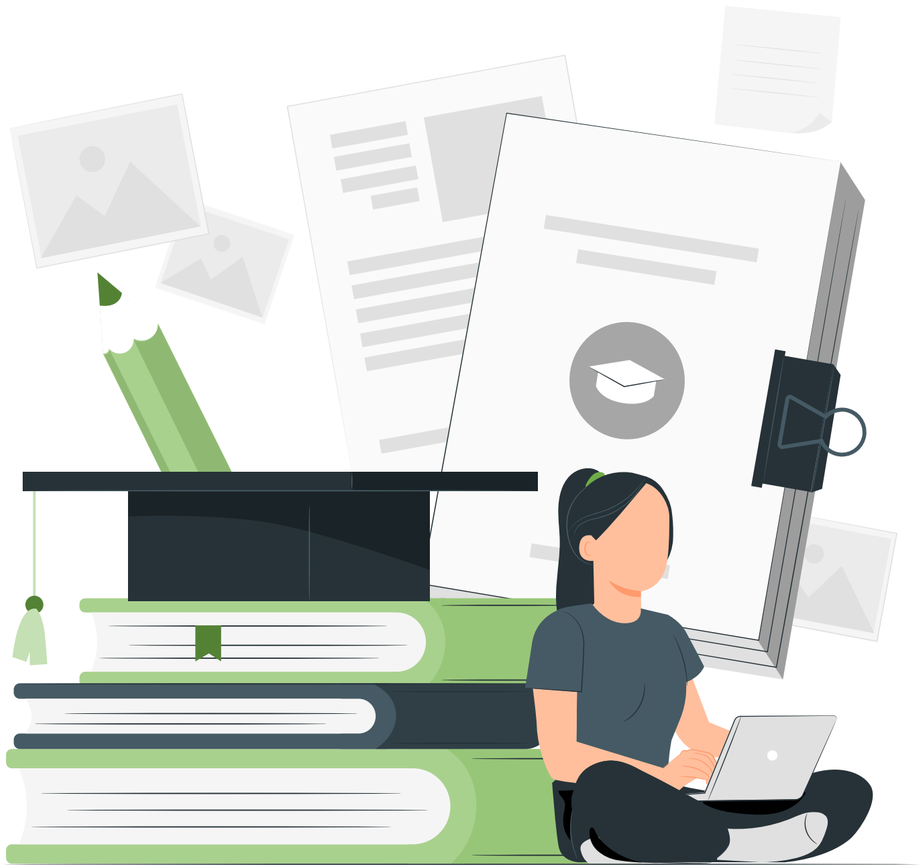
4 Applications

Learning Objectives

By the end of this lecture, students should be able to:

- Understand word embeddings
- Compare Word2Vec, GloVe, and FastText in terms of their approach and strengths
- Understand why static embeddings fail to distinguish between different meanings of the same word
- Describe how self-attention allows each word to "look at" all other words in a sentence
- Load any BERT variant from Hugging Face and extract embeddings

Bibliography



Books:

1. Pilehvar, M. T., & Camacho-Collados, J. (2021). *Embeddings in Natural Language Processing: Theory and Advances in Vector Representations of Meaning*. Morgan & Claypool Publishers.
2. Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed.). Stanford University.

Official Documentation:

1. Gensim Documentation: https://radimrehurek.com/gensim/
2. WordNet: https://wordnet.princeton.edu/
3. GloVe Embeddings: https://nlp.stanford.edu/projects/glove/
4. Hugging Face Transformers Documentation
5. Hugging Face Models Hub

Summary – What We Have Learned So Far

Technique	Description	Example	Applications
Tokenization	Split text into tokens	"I love NLP" → ["I", 'love', "NLP"]	Preprocessing for all NLP tasks
Stemming	Reduce words to their root form (may not be a real word)	"studies" → 'studi' `stemmer.stem("studies")`	Search, recommendation systems
Lemmatization	Reduce words to the base form in the dictionary	"better" → "good" `lemmatizer.lemmatize(word, pos='a')`	Semantic analysis, chatbots
Stop Words	Remove common words with no meaning	"the," "a," "of," "to," "with", "is" (removed)	Classification, clustering
PoS Tagging	Tagging part of speech	"cat" → NOUN, "runs" → VERB	Syntactic understanding
NER	Identifying named entities	"John went to Lisbon" → [John:PERSON, Lisbon:LOC]	Information extraction
Bag of Words	Word count vector	[cat:1, mouse:1, dog:0, ...]	Simple classification

Summary - What We Have Learned So Far

Technique	Description	Example	Applications
TF-IDF	Weight terms by overall importance	Rare words have greater weight	Search, summarization
N-grams	Capture word sequences	“I don't like” (bigram) maintains negation	Sentiment analysis
Dependency Parsing	Identify grammatical relationships between words	“She eats apples” → *eats* → root; *She* → nsubj; *apples* → obj	Syntax analysis, information extraction, question answering
Cosine Similarity	Measure similarity between text vectors	Doc1: [1,1,0], Doc2: [10,10,0] → cosine similarity ≈ 1	Document similarity, search ranking, recommendation systems
Wordnet	Lexical database of synonyms (synsets) and semantic relations	“teacher” → educator, instructor	Semantic analysis, word sense disambiguation, NLP knowledge bases

Summary – What We Have Learned So Far

Limitations of Classical Representations:

- Sparse vectors with very high dimensionality (vocabulary lists with 100k+ words).
- They do not capture semantics or relationships between words.
- No sense of context: ‘right’ (correct) = ‘right’ (direction).
- No treatment of synonyms or polysemy.

Problem: How to automatically combine the best of all worlds?

Word Embeddings

Word Embeddings

What are word embeddings?

Dense and semantic representations of words in a low-dimensional vector space (usually 50 to 300 dimensions).

Revolutionary idea (2013 – Mikolov et al., Google):

"Learn dense vector representations automatically from context"

Why Embeddings Were Revolutionary?

- Instead of **manually** encoding knowledge

We now:

- **Learn meaning automatically from data**

Large corpora → statistical patterns → vector representations.

Word Embeddings

The Distributional Hypothesis

The theoretical basis: *"You shall know a word by the company it keeps" (John Rupert Firth, 1957).*

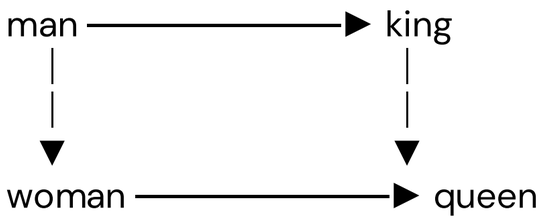
Main Concept: Words that appear in similar contexts tend to have similar meanings.

This is known as: Distributional Hypothesis

Famous Example: Analogies

Vector Arithmetic: Semantic relationships are captured as constant geometric distances.

vector("king") - vector("man") + vector('woman') ≈ vector("queen")



The vector "gender" (man → woman) is the same as (king → queen)!

How are embeddings learned?

Neural networks learn to predict:

- A word from its context

  OR

- Context from a word

During training, vectors are adjusted to reduce prediction error.

Word Embeddings

➡ From Sparse to Dense Vectors

A representation where EACH word is a vector of, typically, 100 to 300 real numbers.

"cat" = [0.21, -0.45, 0.87, 0.12, -0.33, 0.54, -0.78, 0.29, ...] (300 numbers)

"feline" = [0.19, -0.42, 0.85, 0.15, -0.30, 0.52, -0.75, 0.31, ...]

"car" = [-0.65, 0.23, -0.12, 0.91, 0.45, -0.67, 0.33, -0.28, ...]

What does each number represent? Nobody knows exactly! They are latent characteristics learned by the neural network.

Could they represent: size? speed? animality? formality? color?

The model "invents" dimensions that are useful for predicting contexts.

💡 **Important:** The numbers themselves don't matter; what matters are the RELATIONSHIPS between vectors!

Word Embeddings

From Sparse to Dense Vectors

Problems with Bag of Words and TF-IDF:

- High dimensional
- Sparse
- No semantics
- No synonym understanding
- No similarity awareness

Example:

car ≠ automobile

BoW vector for vocabulary size 10,000:

[0,0,0,1,0,0,0,0,0,0,0,...]

Embedding vector (300D):

[0.21, -0.44, 0.87, ..., 0.13]

- Compact
- Dense
- Semantic

The contrast: While TF-IDF and BoW creates giant vectors full of zeros (sparse), embeddings create small vectors full of decimal numbers (dense).

Dense vectors (e.g., 300 dimensions) vs. sparse vectors (50k dimensions)

Word Embeddings

What are word embeddings?

Dense and semantic representations of words in a low-dimensional vector space (usually 50 to 300 dimensions).

STATIC

(one word = one vector)

Word2Vec

GloVe

FastText

Static embeddings remain fixed once learned; that is, they cannot be adjusted or modified for a specific task.

CONTEXTUAL

(one word = multiple vectors)

BERT

GPT

ELMo

Dynamic embeddings can be adjusted for specific tasks, adapting to the specific nuances of the task and the current context

Word Embeddings

➡ Word2Vec - The Revolution (Google, 2013)

Word2Vec: *"Tell me who you hang out with, and I'll tell you who you are"*

What it is: A simple neural network model (one hidden layer) for learning embeddings.

The two architectures:

- CBOW (Continuous Bag of Words): Attempts to predict the central word based on the context (the neighbors).
- Skip-gram: Attempts to predict the context (the neighbors) based on a central word.

⚠ Skip-gram usually works better for rare words.

Word Embeddings

➡ GloVe (Global Vectors for Word Representation)

Source: Stanford (2014).

Difference from Word2Vec: While Word2Vec looks at local text windows, GloVe looks at the global co-occurrence matrix of the entire corpus.

Strengths:

- ✓ Uses global statistics
- ✓ Stable training
- ✓ Good linear structure

Limitations:

- ✗ Static embeddings
- ✗ Large co-occurrence matrix needed

Word Embeddings

➡ FastText

Source: Meta Platforms (2016).

Difference: Instead of looking at the whole word, it looks at n-grams of characters (subwords).

Word: **"playing"**
Character 3-grams: pla, lay, ayi, yin, ing

Strengths:

- ✓ Handles rare words
- ✓ Handles morphology (play, playing, played, player - share subword components → similar vectors.
- ✓ Handles OOV (Out-of-Vocabulary): New word unseen during training is still computable via n-grams.

Word Embeddings

Word2Vec with Gensim

Tokenization with Gensim:

```python
from gensim.utils import simple_preprocess
from gensim.models import Word2Vec

# Corpus de exemplo
corpus = [
    "O gato preto dorme no sofá da sala",
    "O cachorro late para o gato preto",
    "Gatos e cachorros são animais domésticos",
    "A sala tem um sofá confortável",
    "Animais selvagens vivem na floresta"
]

# Tokenização
tokenized_corpus = [simple_preprocess(sentence) for sentence in corpus]
print(tokenized_corpus[0])
# Saída: ['o', 'gato', 'preto', 'dorme', 'no', 'sofá', 'da', 'sala']
```

⚠ *simple_preprocess* already converts to lowercase and removes punctuation

Word Embeddings

Word2Vec with Gensim

Training the Word2Vec Model:

```
from gensim.models import Word2Vec

# Treino
modelo = Word2Vec(
    sentences=tokenized_corpus,  # corpus tokenizado
    vector_size=100,              # dimensão do embedding
    window=5,                     # tamanho da janela de contexto
    min_count=1,                  # frequência mínima da palavra
    workers=4,                    # threads para treino paralelo
    sg=0,                         # 0=CBOW, 1=Skip-gram
    epochs=5                      # número de iterações
)

print("Modelo treinado!")
print(f"Vocabulário: {len(modelo.wv.key_to_index)} palavras")
```

Word Embeddings

Word2Vec with Gensim

Exploring vocabulary:

```
# Aceder ao vocabulário
palavras = list(modelo.wv.key_to_index.keys())
print("Primeiras 10 palavras:", palavras[:10])

# Verificar se palavra existe
if 'gato' in modelo.wv:
    print("'gato' está no vocabulário")

# Tamanho do vetor
print(f"Dimensão do embedding: {modelo.wv.vector_size}")
```

OUTPUT:
Primeiras 10 palavras: ['o', 'gato', 'preto', 'dorme', 'no', 'sofá', 'da', 'sala', 'cachorro', 'late']
'gato' está no vocabulário
Dimensão do embedding: 100

Word Embeddings

➡ Word2Vec with Gensim

Obtaining word vectors:

```
# Obter o vetor de uma palavra
vetor_gato = modelo.wv['gato']
print(f"Vetor 'gato' (primeiras 10 dimensões):")
print(vetor_gato[:10])
print(f"Shape: {vetor_gato.shape}")

# Comparação com one-hot
print(f"\nOne-hot teria dimensão: {len(modelo.wv.key_to_index)}")
print(f"Embedding tem: {modelo.wv.vector_size} dimensões")
```

Key concept: Dense vs. sparse vectors

One-hot: 10k dimensions (99.9% zeros)

Embedding: 100 dimensions (dense, 100% non-zero)

Word Embeddings

➡ Word2Vec with Gensim

Finding Similar Words:

```
# Palavras mais similares a 'gato'
similares = modelo.wv.most_similar('gato', topn=5)
print("Palavras similares a 'gato':")
for palavra, score in similares:
    print(f"  {palavra}: {score:.4f}")

# Similaridade entre duas palavras
sim = modelo.wv.similarity('gato', 'cachorro')
print(f"\nSimilaridade 'gato' vs 'cachorro': {sim:.4f}")
```

Interpretation:

- **Score close to 1** → high semantic similarity
- **Score close to 0** → unrelated words
- **Negative scores are rare**

Word Embeddings

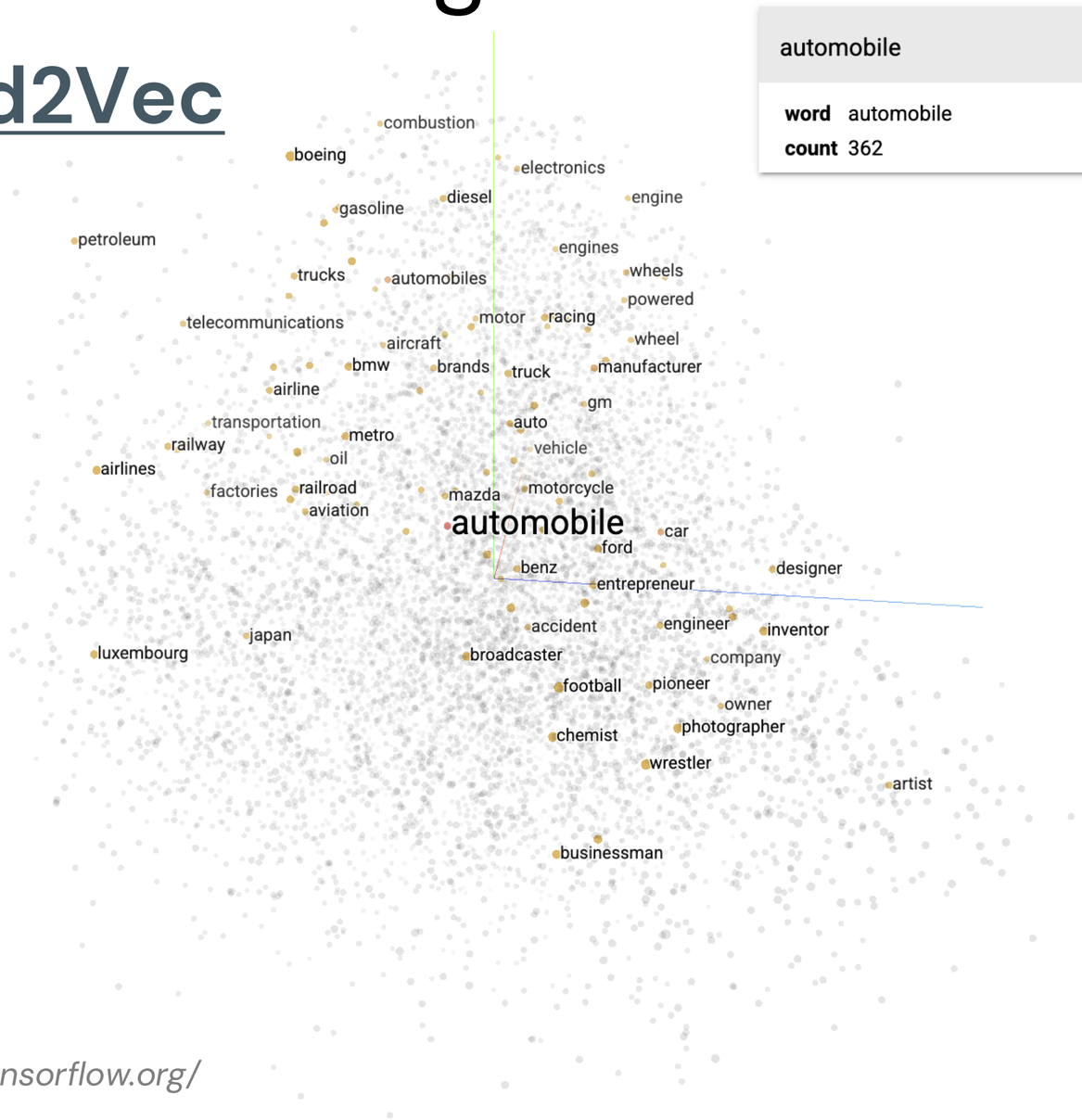
➡ Word2Vec with Gensim

Critical parameters

Parameter	Typical Values	Impact
vector_size	100-300	>300: marginal gain, more memory
window	5-10	Larger window → more context/topic
min_count	1-10	Remove rare words (noise)
sg	0 (CBOW) / 1 (Skip-gram)	Skip-gram better for rare words
negative	5-20	Negative sampling speeds up training
epochs	5-50	More epochs → overfitting possible

Word Embeddings

Word2Vec



automobile

word automobile
count 362

Nearest points in the original space:

car	0.493
automobiles	0.503
manufacturer	0.533
motors	0.537
motorcycle	0.585
aircraft	0.588
benz	0.593
auto	0.593
mercedes	0.599
motor	0.600
cars	0.616
airplane	0.619
truck	0.620
diesel	0.645

https://projector.tensorflow.org/

Word Embeddings

➡ GloVe with Gensim

Using Pre-trained Embeddings

```
import gensim.downloader as api

# Listar modelos disponíveis
print(list(api.info()['models'].keys()))

# Carregar modelo GloVe (Twitter, 25 dimensões)
# Outras opções: 'glove-twitter-50', 'glove-wiki-gigaword-50', 'fasttext-wiki-news-subwords-300'
modelo_glove = api.load("glove-twitter-25")

# Usar o modelo
vetor = modelo_glove['computer']
similares = modelo_glove.most_similar('computer', topn=5)

print("Palavras similares a 'computer':")
for palavra, score in similares:
    print(f"  {palavra}: {score:.4f}")
```

Advantage: Models trained on billions of words

Word Embeddings

➡ GloVe with Gensim

Famous Example – Analogies

```
# Analogia: rei - homem + mulher = ?
resultado = modelo_glove.most_similar(
    positive=['woman', 'king'],
    negative=['man'],
    topn=3
)
print("Rei - Homem + Mulher:")
for palavra, score in resultado:
    print(f"  {palavra}: {score:.4f}")

# Analogia: Paris - França + Itália = ?
resultado = modelo_glove.most_similar(
    positive=['Italy', 'Paris'],
    negative=['France'],
    topn=3
)
print("\nParis - França + Itália:")
for palavra, score in resultado:
    print(f"  {palavra}: {score:.4f}")
```

Expected output: ‘queen’, 'Rome'

Word Embeddings

➡ FastText with Gensim

Handling Out-of-Vocabulary Words (OOV)

```
# Problema: palavra não está no vocabulário
if 'deeplearning' in modelo_glove:
    vetor = modelo_glove['deeplearning']
else:
    print("Palavra 'deeplearning' não encontrada!")

# Solução com FastText (subword information)
from gensim.models import FastText

# FastText consegue vetores para palavras desconhecidas
modelo_fasttext = FastText(
    sentences=tokenized_corpus,
    vector_size=100,
    window=5,
    min_count=1
)

# Mesmo se não viu a palavra, pode inferir
vetor = modelo_fasttext.wv['deeplearning']  # Funciona!
```

Word Embeddings

➡ Summary

- GloVe → statistical matrix factorization
- FastText → compositional word representation
- Word2Vec → predictive local model

Model	Context	OOV	Base
Word2Vec	Local Window	✗	Words
GloVe	Global Corpus	✗	Co-ocorrência
FastText	Local Window	✓	Subwords

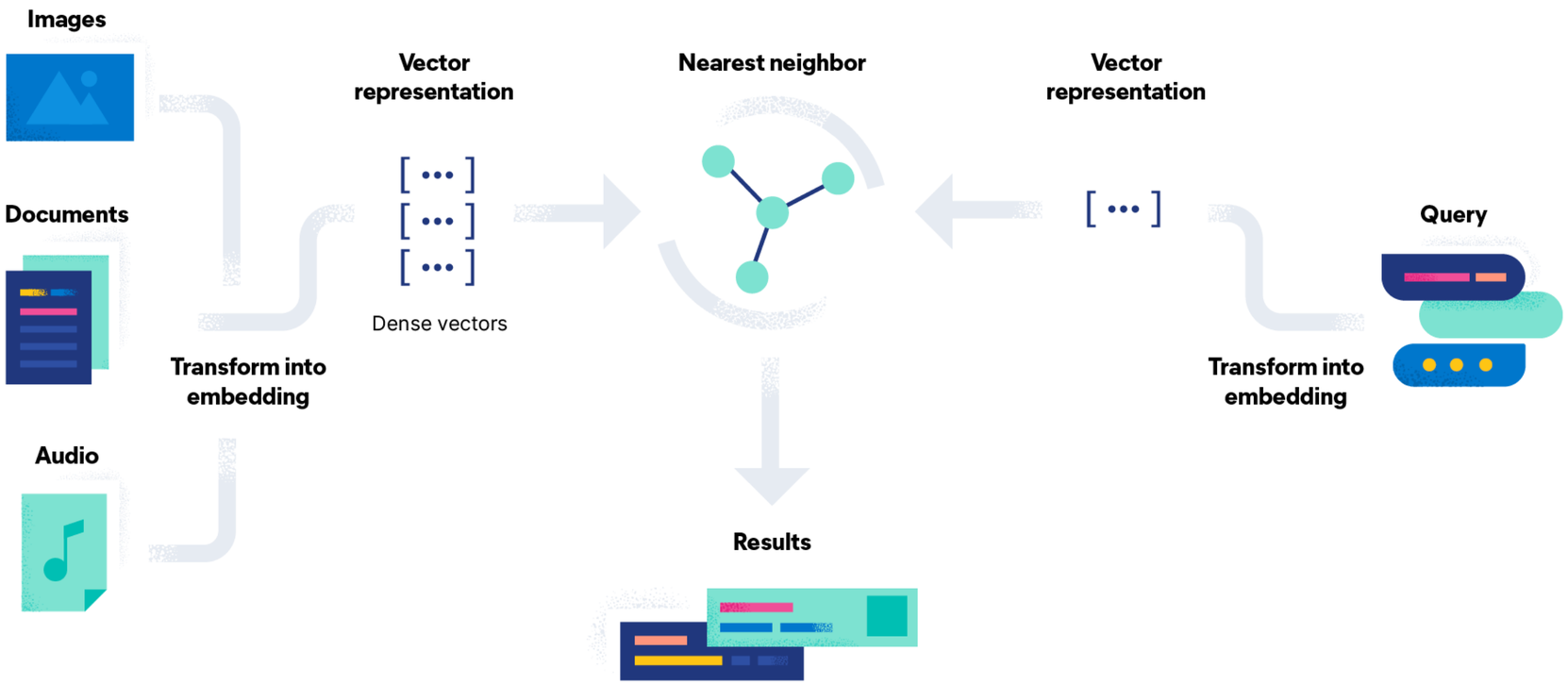
All produce **static embeddings**.

Why use Static Embeddings today?

- **Efficiency:** They are extremely fast. You load a .txt or .bin file and do a word lookup.
- **Low Cost:** They do not require powerful GPUs to be used in inference.
- **Use:** Excellent for simple classification tasks or when the vocabulary is very specific (e.g., medical terms).

Word Embeddings

➡ Applications: Search Engines



- Sentiment Analysis and Text Classification
- Recommendation Systems
- Machine Translation and Multilingual Models
- Question and Answer (QA) Systems and Chatbots

Word Embeddings

Limitations

Static Embeddings → The fundamental **problem**: A word has the SAME UNIQUE VECTOR , regardless of context!

The mouse ate the cheese. (mouse – animal) vs. *Click the mouse to continue.* (mouse – computer device)

Q: Does the word ”mouse" mean the same thing in the sentences? **A:** NO!

Both meanings map to the same point in the vector space — the average of the contexts.

- ✗ Does not capture polysemy (multiple meanings)
- ✗ Does not adapt to the local context

We need CONTEXTUAL representations!

- ✓ The same token can have different representations
- ✓ The representation depends on ALL other words in the sentence
- ✓ Context modifies the meaning

Contextual Embeddings

➡ ELMo, BERT and beyond

ELMo (2018): embeddings generated by bidirectional LSTMs — representation depends on context

BERT (2018): uses Transformers; the representation of each word changes depending on the sentence

Dynamic word embedding is responsible to:

- capture the semantic meaning of words
- reduce dimensionality
- add contextual information
- promote efficient learning by transferring linguistic knowledge via pre-trained embeddings.

Before the Transformers

➡ The Problem of RNNs/LSTMs

How RNNs processed text:

”The" → [h1] → ”black" → [h2] → ”cat" → [h3] → "dorme" → [h4]

↑	↑	↑
Information of “The” flows forward	Information of “The” + “black” flows forward	Information of the whole context flows forward

Problems:

- ❌ Sequential processing (slow, not parallelizable)
- ❌ Information degrades into long sentences
- ❌ Difficulty with distant dependencies

The Attention Mechanism

➡ The Central Idea "Pay Attention to What Matters"

Analogy: You are at a party full of people talking. To understand what your friend is saying, you pay attention to their voice and ignore the rest.

In text processing:

"The mouse clicked the icon on the screen."

To understand "mouse":

- **HIGH attention** to "icon" and "screen" (relevant context)

- **LOW attention** to "on" (functional words) and "The" (article)

Result **mouse = computer device**, not animal.

Self-Attention Mechanism

➡ The Central Idea "Pay Attention to What Matters"

Each word looks at the other words and asks: **“Which of you helps me understand my meaning?”**

"The black cat sleeps"

Step 1: Each word becomes a vector:
The sentence is converted into embeddings (numeric vectors).

”The" → [0.2, 0.1, 0.3]
”black" → [0.6, 0.9, 0.5]
”cat" → [0.7, 0.4, 0.6]
”sleeps" → [0.5, 0.3, 0.8]

Step 2: Create Query, Key, Value vectors
- **Query** (what am I looking for?)
- **Key** (what do I offer?)
- **Value** (what information do I have?)

These are produced using **learned weight matrices.**

Query = embedding × Wq
Key = embedding × Wk
Value = embedding × Wv

The matrices **Wq, Wk, Wv** are learned during training.

Self-Attention Mechanism

➡ The Central Idea "Pay Attention to What Matters"

Each word looks at the other words and asks: **“Which of you helps me understand my meaning?”**

"The black cat sleeps"

Step 3: Compute similarity (attention score):

For a word like ”cat", we compare its Query with the Keys of all words using a dot product.

Query(cat) · Key(The) = 0.2 (low attention - article)
Query(cat) · Key(black) = 1.6 (high attention - characteristic)
Query(cat) · Key(sleeps) = 1.2 (medium attention - action)

Step 4: Normalize with Softmax

The model converts them into attention weights between **0** and **1** using softmax.

softmax(0.2, 1.6, 1.2) → 0.1 , 0.5 , 0.4

Now they behave like **importance weights**.

Step 5: Weighted combination of Values
New representation of "cat" = weighted sum of Values**: 0.1 * Value(The)+ 0.5 * Value(black)+ 0.4 * Value(sleeps)**
(a lot of "black", some of "sleeps", little of ”The")
Result: "cat" now contains information about being black and sleep!

Self-Attention Mechanism

Heat Map of Attention

WORD THAT RECEIVES ATTENTION

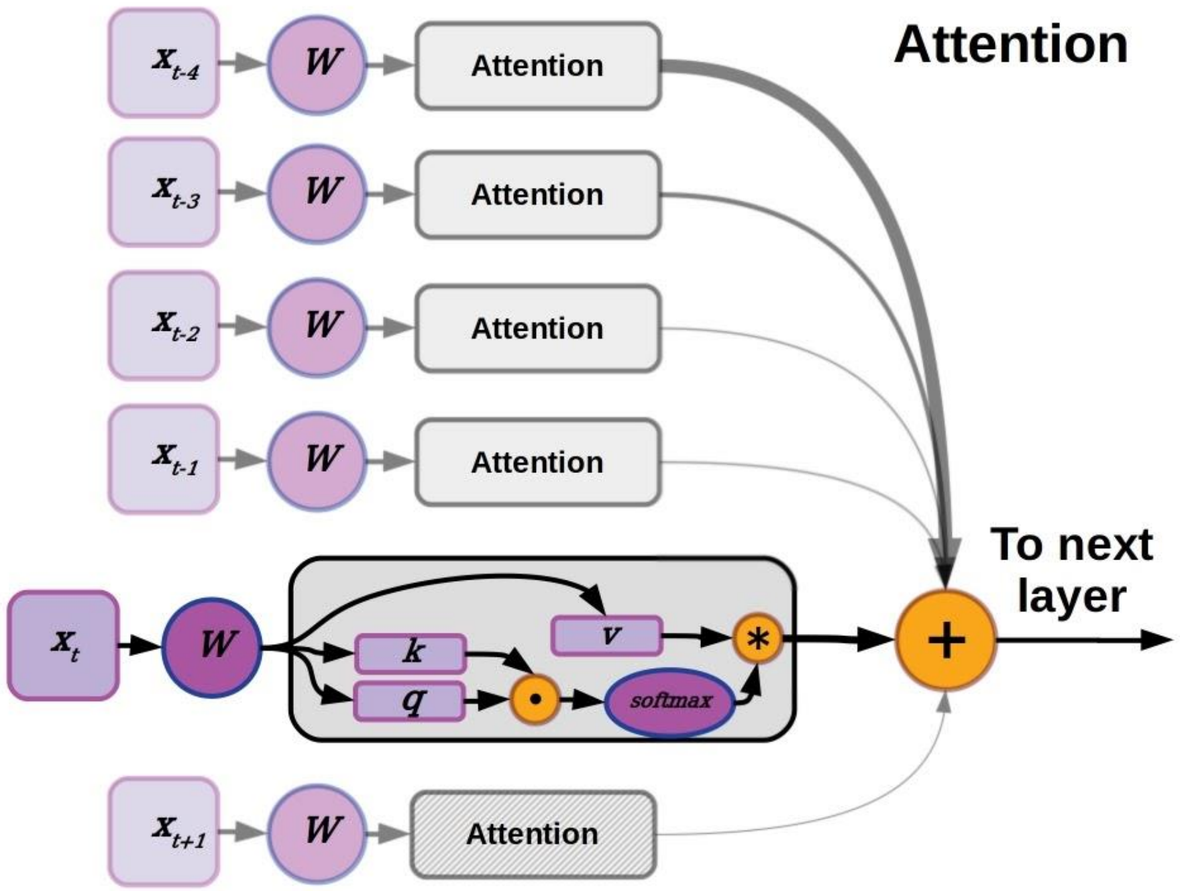
WORD	The	black	cat	sleeps
The	0.9	0.03	0.04	0.03
black	0.01	0.5	0.3	0.19
cat	0.02	0.5	0.4	0.08
sleeps	0.01	0.02	0.4	0.57

Interpretation:
- "cat" pays attention to itself (0.4) and to "black" (0.5)
- "black" pays attention to "cat" (0.3) and "sleeps" (0.19)
- Each line adds 1 (attention distribution)

Each word is represented by a weighted combination of ALL the words!

Transformers

➡ Architecture



OUTPUT

↑

[DECODER]

↑

[ATTENTION]

↑

[ENCODER]

↑

INPUT

Encoder: Understand the text

Decoder: Generate text

Contextual Embeddings

➡ BERT (Bidirectional Encoder Representations from Transformers)

Google AI (2018) – The Model that Understands Context

BERT leverages a **transformer-based neural network** to understand and generate human-like language.

- **Architecture:** Only the Transformer ENCODER
- **Objective:** To create RICH and CONTEXTUAL text representations
- **Key innovation:** BIDIRECTIONALITY

Previous models (ELMo, GPT-1) were **unidirectional**:

→ → → → → → → → → → → → → (they only look to the left)

BERT is **bidirectional**:

← ← ← ← ← ← ← ← ← ← ← ← ←

→ → → → → → → → → → → → →. (each word sees ALL the others)

⚠ **Why does this matter?** To understand "mouse" in the sentence, you need to see the words from both sides!

Contextual Embeddings

➡ BERT Architecture

Transformer Architecture: Transformers are highly parallelizable and efficient for capturing long-range dependencies in text.

Bidirectional Encoding: BERT has a bidirectionality feature that enables analyze the context of each word by considering all the words in the sentence.

Pre-training and Fine-tuning: BERT is capable of learning rich and contextualized word representations as it is pre-trained on a massive corpus of text data. And we can perform fine-tuning on this pre-trained data as per our requirement which makes BERT highly adaptable and effective for various NLP tasks.

Multi-Layer Stacking: BERT consists of stacked transformers i.e. it has multiple layers of transformers stacked on top of each other. This feature enables BERT to capture more complex contextual information.

Contextual Embeddings

➡ BERT Training

TRAINING OBJECTIVES:

- MLM (Masked Language Model) - BERT is trained to predict masked words within a sentence. This forces the model to understand the context of words in relation to their surroundings.

- NSP (Next Sentence Prediction) - designed to teach the model sentence-level relationships by classifying whether a segment B follows segment A. It enables BERT to understand context and flow for tasks like Q&A and inference, although it is often excluded in later models like RoBERTa.

What's the difference?

Masked LM helps BERT to understand the context within a sentence and Next Sentence Prediction helps BERT grasp the connection or relationship between pairs of sentences.

Contextual Embeddings

➡ BERT Training – MLM

BERT is trained with an ingenious task:

Original sentence: "The black cat sleeps on the sofa"

Masked sentence: "The [MASK] cat sleeps on the sofa"

BERT needs to predict: "black"

How does this create contextual embeddings? To predict the masked word, BERT needs to:

1. Understand that "cat" is an animal
2. Understand that animals can have colors
3. Using the context of the entire sentence
4. Know which colors are common for cats

By doing this MILLIONS of times, BERT learns to create representations that capture ALL the context!

Contextual Embeddings

➡ GPT - Generative Pre-trained Transformer

OpenAI (2018) - The Model that Generates Text

- **Architecture:** Only the Transformer DECODER
- **Objective:** Generate text fluently
- **Training task:** Predict the NEXT word

Training text: "*The black cat sleeps*"

Step 1: "The" → predict "cat"
Step 2: "The cat" → predict "black"
Step 3: "The black cat" → predict "sleeps"
Step 4: "The black cat sleeps" → predict <end>

Crucial difference from BERT:

- **BERT:** sees everything and fills in gaps → UNDERSTANDING
- **GPT:** sees the past and predicts the future → GENERATION

Contextual Embeddings

➡ BERT vs. GPT

Feature	BERT	GPT
Architecture	Transformer **Encoder**	Transformer **Decoder**
Context	**Bidirectional** (looks at left and right context)	**Unidirectional** (predicts next word from previous words)
Training Task	**Masked Language Model** (predict missing words)	**Next Word Prediction**
Strength	Deep **language understanding**	**Text generation**
Typical Tasks	Sentiment analysis, question answering, NER	Chatbots, writing, summarization, coding

Contextual Embeddings

➡ Hugging Face

Hugging Face is a company that created the most widely used library for working with transformer models: the transformers library.

💡 **Think of it as the "App Store" for NLP models**

Hugging Face hosts over 100,000 pre-trained models:

- **Different architectures:** BERT, GPT, RoBERTa, XLNet, DistilBERT
- **Multilingual models:** Versions for Portuguese, Vietnamese, etc.
- **Specialized models:** BioBERT (medical), LegalBERT, etc.

Hugging Face

➡ Loading ANY BERT Variant from Hugging Face

```
# First, let's install/upgrade the necessary libraries
!pip install transformers datasets accelerate torch sentence-transformers
```

```
from transformers import AutoTokenizer, AutoModel, AutoConfig
import torch
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt

print("Setup complete!")
print(f"Transformers version: {transformers.__version__}")
print(f"Torch version: {torch.__version__}")
```

Hugging Face

➡ Loading ANY BERT Variant from Hugging Face

```
# Example 1: The Original BERT (Base, Uncased)
print("="*70)
print("1. ORIGINAL BERT (Google)")
print("="*70)

model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

print(f"Model: {model_name}")
print(f"Vocabulary size: {tokenizer.vocab_size}")
print(f"Model configuration: {model.config}")

# Let's see it in action
text = "I went to the bank"
inputs = tokenizer(text, return_tensors="pt")
outputs = model(**inputs)

print(f"\nInput text: '{text}'")
print(f"Input tokens: {tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])}")
print(f"Output shape: {outputs.last_hidden_state.shape}")  # [batch, tokens, hidden_size]
print(f"Hidden size: {model.config.hidden_size} dimensions")
```

Hugging Face

➡ Loading ANY BERT Variant from Hugging Face

```
# Example 2: BERT Multilingual (for Portuguese!)
print("\n" + "="*70)
print("2. BERT MULTILINGUAL (works for Portuguese)")
print("="*70)

model_name = "bert-base-multilingual-cased"  # Supports 104 languages including Portuguese
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

# Test with Portuguese text
portuguese_text = "Eu fui ao banco depositar dinheiro"
inputs = tokenizer(portuguese_text, return_tensors="pt")
outputs = model(**inputs)

print(f"Model: {model_name}")
print(f"Text: '{portuguese_text}'")
print(f"Tokens: {tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])}")
print(f"Embedding for 'banco' (contextual!): {outputs.last_hidden_state[0, 4, :10]}")  # First 10 values
```

Hugging Face

➡ Loading ANY BERT Variant from Hugging Face

```
# Example 3: DistilBERT (Faster, lighter version)
print("\n" + "="*70)
print("3. DISTILBERT (60% faster, 40% smaller, 95% performance)")
print("="*70)

model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

print(f"Model: {model_name}")
print(f"Number of layers: {model.config.n_layers} (vs 12 in BERT base)")
print(f"Hidden size: {model.config.hidden_size}")
```

Hugging Face

➡ Loading ANY BERT Variant from Hugging Face

```
# Example 4: RoBERTa (Robustly optimized BERT)
print("\n" + "="*70)
print("4. ROBERTA (Facebook's improved BERT)")
print("="*70)

model_name = "roberta-base"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

print(f"Model: {model_name}")
print(f"Vocabulary size: {tokenizer.vocab_size}")
print(f"Architecture: {model.config.architectures}")
```

Hugging Face

➡ Loading ANY BERT Variant from Hugging Face

```
# Example 5: ALBERT (Lightweight BERT)
print("\n" + "="*70)
print("5. ALBERT (A Lite BERT - parameter sharing)")
print("="*70)

model_name = "albert-base-v2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

print(f"Model: {model_name}")
print(f"Parameter sharing across {model.config.num_hidden_layers} layers")
print(f"Embedding size: {model.config.embedding_size}")
```

Hugging Face

➡ Comparing Embeddings from Different Models

```
# Let's compare how different models represent the same word
print("\n" + "="*70)
print("COMPARING EMBEDDINGS ACROSS MODELS")
print("="*70)

test_sentences = [
    "I went to the bank to deposit money",
    "The river bank was muddy"
]

models_to_test = [
    "bert-base-uncased",
    "distilbert-base-uncased",
    "roberta-base"
]

...
```

Hugging Face

➡ Comparing Embeddings from Different Models

```
...

def get_word_embedding(model_name, sentence, word_index):
    """Get embedding for a specific word in a sentence"""
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModel.from_pretrained(model_name)

    inputs = tokenizer(sentence, return_tensors="pt")
    with torch.no_grad():
        outputs = model(**inputs)

    # Get embedding at word_index (plus 1 for [CLS] token)
    return outputs.last_hidden_state[0, word_index+1].numpy()

...
```

Hugging Face

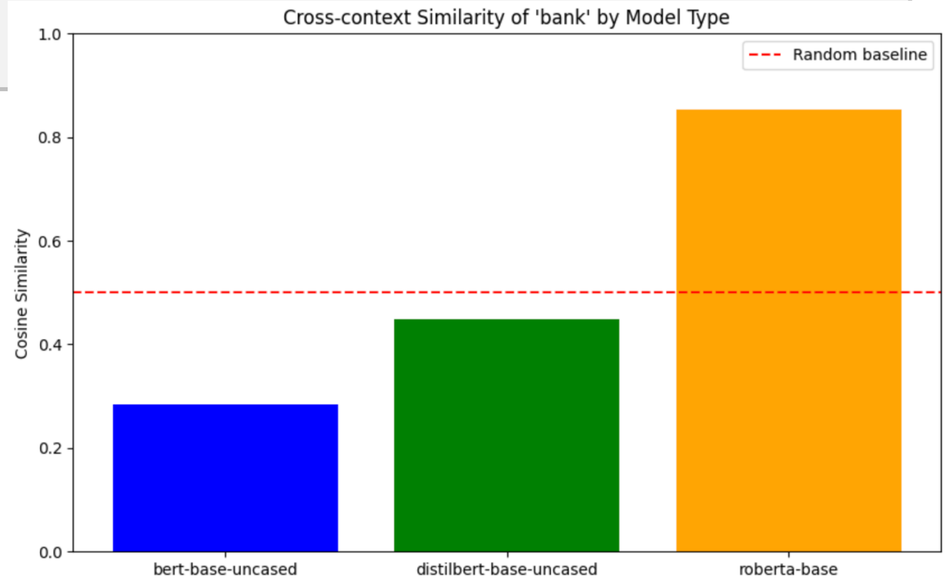
➡ Comparing Embeddings from Different Models

```
...

results = {}
for model_name in models_to_test:
    print(f"\n{model_name}:")
    model_results = []

    for sent in test_sentences:
        # Get embedding for "bank" (index 4 in both sentences after tokenization)
        emb = get_word_embedding(model_name, sent, 4)
        model_results.append(emb)

    # Compare the two embeddings
    similarity = cosine_similarity([model_results[0]], [model_results[1]])[0][0]
    print(f"  Similarity between 'bank' in two contexts: {similarity:.4f}")
    results[model_name] = similarity
```

Hugging Face

➡ Comparing Embeddings from Different Models

```
# Visualize comparison
plt.figure(figsize=(10, 6))
plt.bar(results.keys(), results.values(), color=['blue', 'green', 'orange'])
plt.ylim(0, 1)
plt.title("Cross-context Similarity of 'bank' by Model Type")
plt.ylabel("Cosine Similarity")
plt.axhline(y=0.5, color='r', linestyle='--', label='Random baseline')
plt.legend()
plt.show()
```



Summary

- Embeddings overcome limitations of BoW/TF-IDF
- Word2Vec, GloVe, FastText: dense representations
- Static vs. contextual embeddings
- Attention and Transformers: the basis of LLMs
- BERT and GPT: models that understand context

Kahoot

Goal

- To **reward and compensate students** who actively attend and participate in classes
- To reinforce key concepts in an engaging and competitive way

Rules

- Only students **physically present in class** may participate
- No external help (notes, phones, AI tools, or collaboration)
- Any form of cheating results in **disqualification**

Scoring & Rewards

🥇 **1st place**: +0.5 bonus points

🥈 **2nd place**: +0.3 bonus points

🥉 **3rd place**: +0.2 bonus points

(Bonus points are added to the continuous assessment component.)

⚠️ Bonus points are **non-transferable** and **cannot exceed course limits**

Practical Application

➡ **Worksheet PL6**

Blackboard